

ADC Acquisition

ADC Acquisition

1. Learning Objectives
 - ADC Principle
 - ADC Basic Parameters
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. ADC Parameter Configuration
 3. ADC Acquisition Channel Configuration
 4. Serial Parameter Configuration
 5. Write Program
 6. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Understand basic ADC knowledge.
2. Learn the basic use of ADC and display the acquired values on the serial assistant.

ADC Principle

ADC (analog to digital converter) is used to convert analog signals (such as voltage) into digital signals. Analog signals change continuously, while digital signals are discrete binary numbers. ADC converts analog signals to digital data through sampling and quantization, so that processors or microcontrollers can perform subsequent processing. According to conversion principles, they are mainly divided into successive approximation, dual-slope, and voltage-to-frequency conversion types.

MSPM0G3519 uses successive approximation (SAR) ADC, which is a common ADC working principle. Its basic idea is to gradually approach the digital representation of the input signal by comparing the size relationship between the analog signal and the reference voltage. In the successive approximation ADC, the input signal and reference voltage are compared through a differential amplifier to generate a differential voltage. Then, this differential voltage is input to a gradually approaching digital quantizer, which compares it with a series of reference voltages at each approximation stage. At each approximation stage, the quantizer compares the input signal with an intermediate voltage point and selects a higher or lower reference voltage as the reference for the next approximation stage based on the comparison result. This process continues until the quantizer finally approaches a digital output value.

ADC Basic Parameters

1. Resolution:

- Resolution represents the accuracy of ADC converter output, usually measured in bits (bit), such as 8-bit, 10-bit, 12-bit, etc. Higher resolution means the ADC can represent more discrete digital values, thereby providing higher accuracy.
- MSPM0G3519** supports 8-bit, 10-bit, and 12-bit resolution. Users can select the appropriate resolution for configuration based on their needs.

2. Sample Rate:

- Sample rate (also called conversion rate) represents the rate at which ADC samples the analog input signal, usually expressed in samples per second (SPS). It indicates how many analog-to-digital conversions the ADC can perform per second.
- MSPM0G3519** has a sample rate of 4MSPS (4 million samples per second), suitable for high-frequency signal acquisition and real-time data processing.

3. Voltage Reference:

- The ADC voltage reference is a reference voltage used to compare with the analog input signal to finally achieve conversion from analog signal to digital signal. The accuracy and stability of the voltage reference are crucial to the ADC conversion accuracy.
- MSPM0G3519 supports three voltage reference configurations:
 - Internally configurable reference voltage: 1.4V and 2.5V dedicated ADC reference voltage (VREF).
 - MCU power supply voltage (VDD) as the reference voltage.
 - External reference voltage provided through VREF+ and VREF- pins. If the voltage reference is not configured, the MCU power supply voltage (VDD) is used as the reference voltage by default.

4. Sampling Range:

- Sampling range represents the voltage range of analog input signals that ADC can acquire, which is closely related to the reference voltage setting. The range is as follows: $V_{REF-} \leq \text{ADC} \leq V_{REF+}$
 - VREF-:** The set reference voltage negative terminal, usually 0V.
 - VREF+:** The set reference voltage positive terminal, determined by software configuration.

These parameters together determine the ADC performance, including its accuracy, response speed, and input voltage range.

2. Hardware Setup

MSPM0G3519 uses a 12-bit successive approximation ADC. It has 17 multiplexed channels for conversion. The 17 external channels each correspond to a pin on the microcontroller. This pin is not fixed. For details, please refer to the pin diagram or data manual.

									A0_1	(Non-IOMUX 1) 0	A	
									COMP0_IN0+	(Non-IOMUX 2) 0	A	
31	B2	G2	47	47	31	79	99	PA27 PINCM60 0x404280ec	PA27	1	IO	SDIO (standard)
									UART3_RX	2	I	
									SPI1_CS1	3	IO	
									TIM8_C1	4	IO	
									TIMA_FAL2	5	I	
									CLK_OUT	6	O	
									RTC_OUT	8	O	
									COMP0_OUT	9	O	
									CAN0_RX	10	I	
									TIM7_C1	11	IO	
									A0_0	(Non-IOMUX 1) 0	A	
									COMP0_IN0-	(Non-IOMUX 2) 0	A	

Table 6-3. Analog to Digital Converter (ADC) Signal Descriptions

SIGNAL NAME	PIN TYPE	DESCRIPTION	RHB PIN	ZAW PIN	YCJ PIN	RGZ PIN	PT PIN	PM PIN	PN PIN	PZ PIN
A0_0	A	ADC0 analog input channel 0	31	B2	G2	47	47	31	79	99
A0_1	A	ADC0 analog input channel 1	30	C2	G3	46	46	30	78	98
A0_2	A	ADC0 analog input channel 2	29	D1	F3	45	45	26	74	94
A0_3	A	ADC0 analog input channel 3	28	E2	F4	44	44	25	73	93
A0_4	A	ADC0 analog input channel 4		C1				27	75	95

Various channel A/D conversions can be configured in **single or sequence conversion** mode.

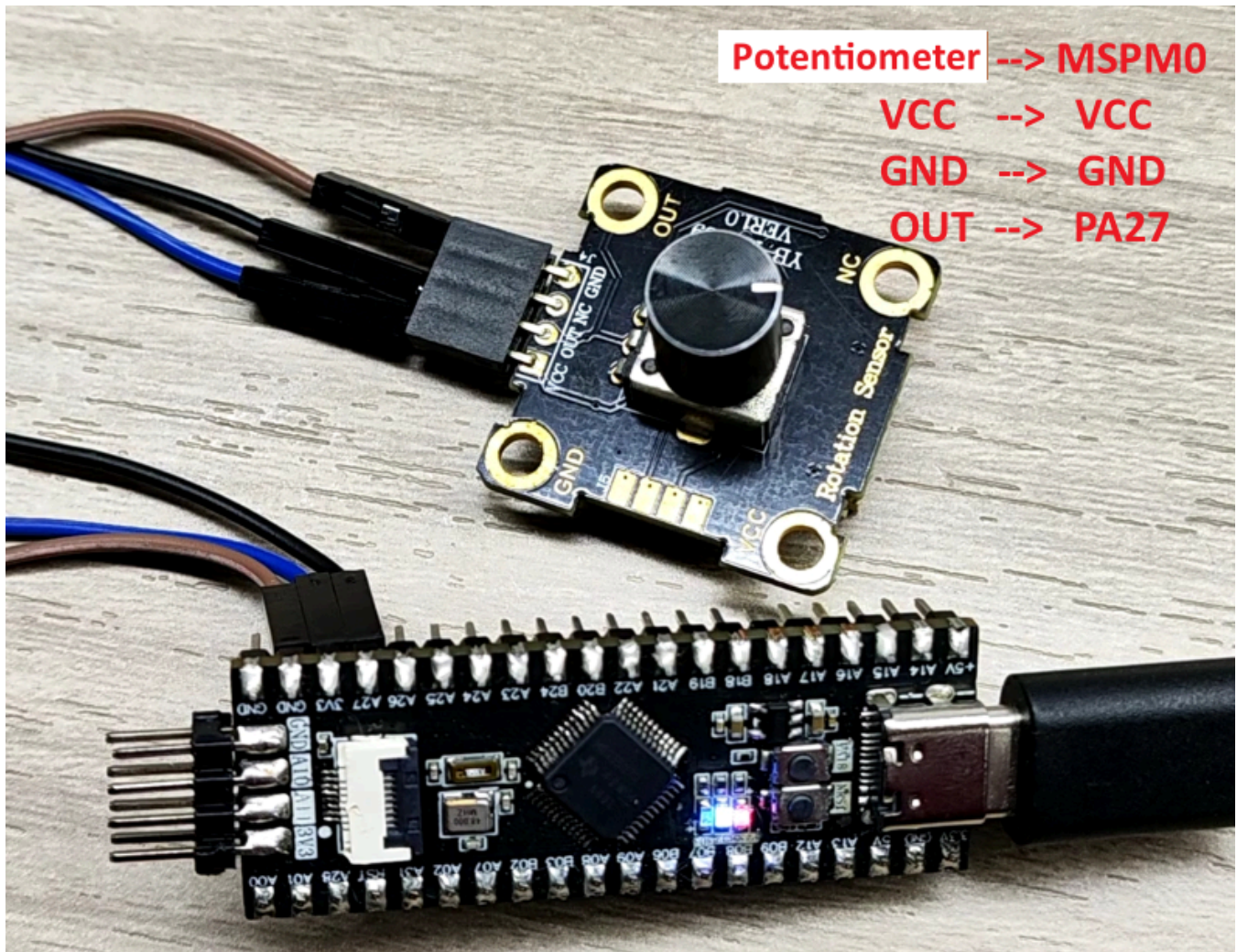
Single Conversion Mode: After each ADC conversion is completed once, the ADC automatically stops and stores the result in the ADC data register.

Repeated Single Conversion Mode: When the ADC completes one conversion, it automatically starts another conversion, continuously performing conversions until stopped by an external trigger or software trigger.

Multi-channel Sequential Single Conversion Mode: Used for sequential conversion of multiple input channels. In this mode, the ADC samples and converts multiple channels sequentially according to the configured channel acquisition order.

Multi-channel Sequential Repeated Conversion Mode: Used for sequential repeated conversion of multiple input channels. In this mode, the ADC performs repeated sampling and conversion of multiple channels sequentially according to the configured channel acquisition order.

This case acquires the PA27 pin voltage through ADC. The potentiometer module and Dupont wires needed for the ADC acquisition experiment need to be purchased separately. You can also use other input voltage methods.

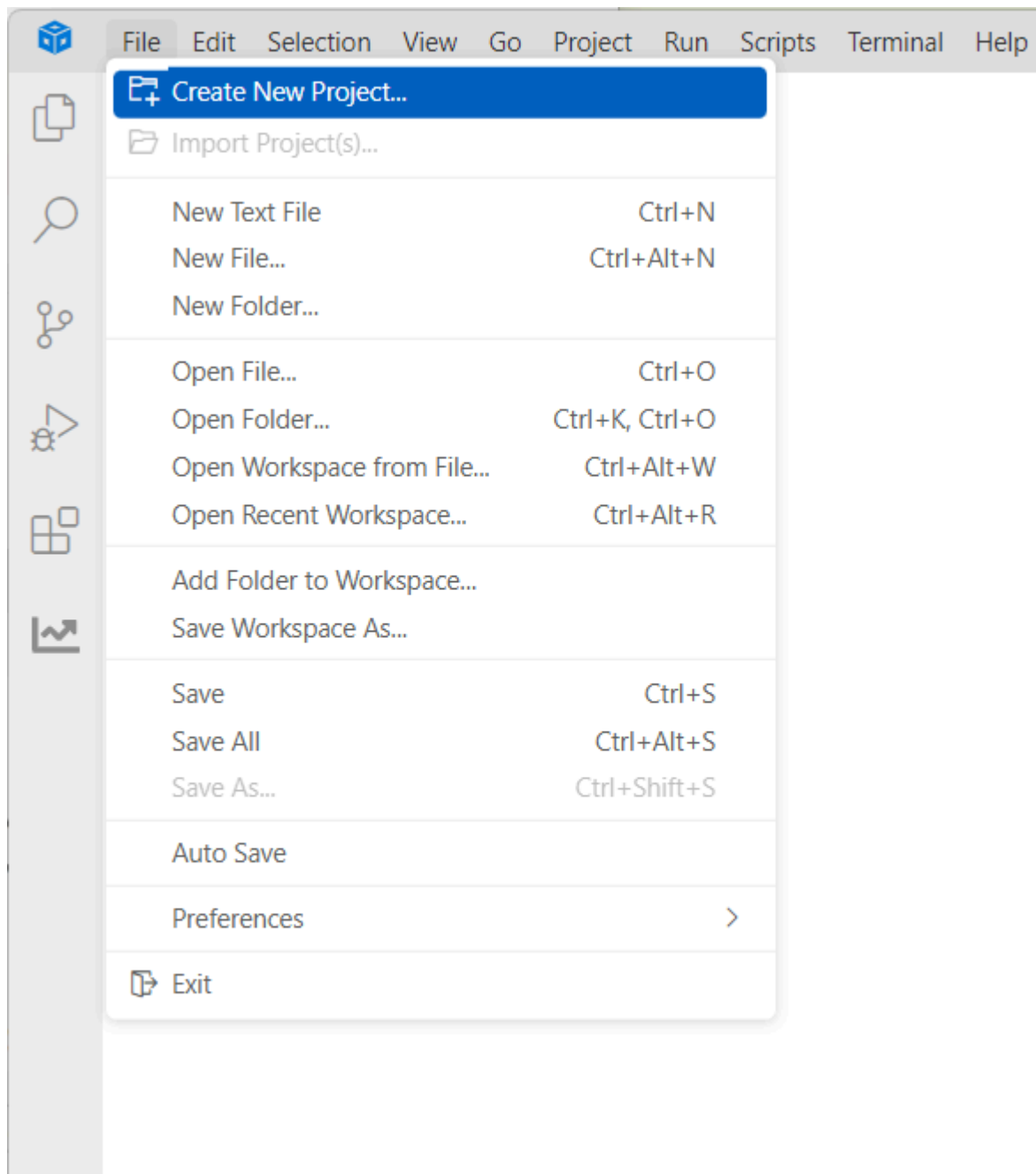


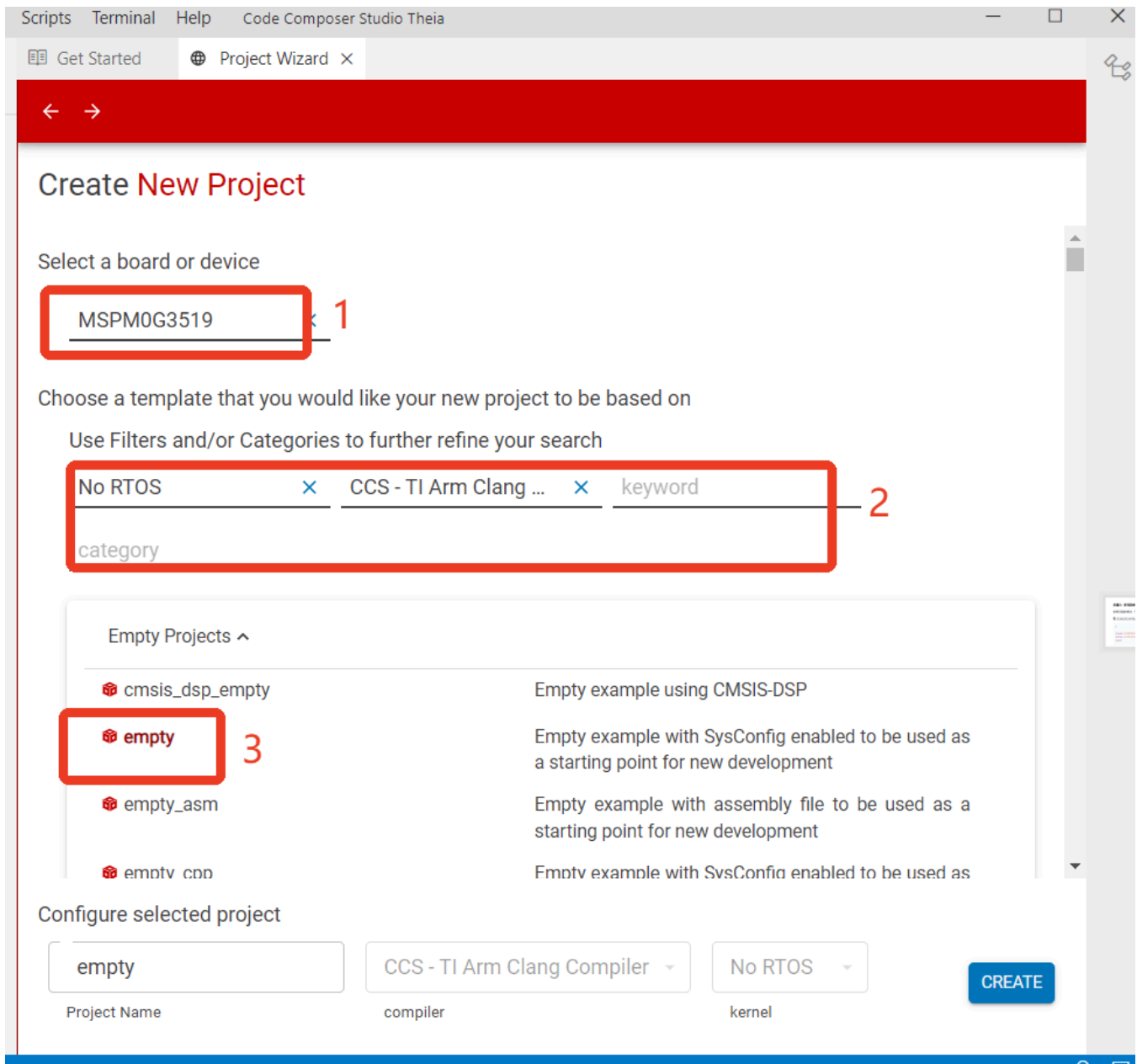
3. Experiment Steps

This case uses PA27 pin voltage acquisition as the experimental case. Through ADC acquisition of its voltage changes, implement serial port printing of voltage status.

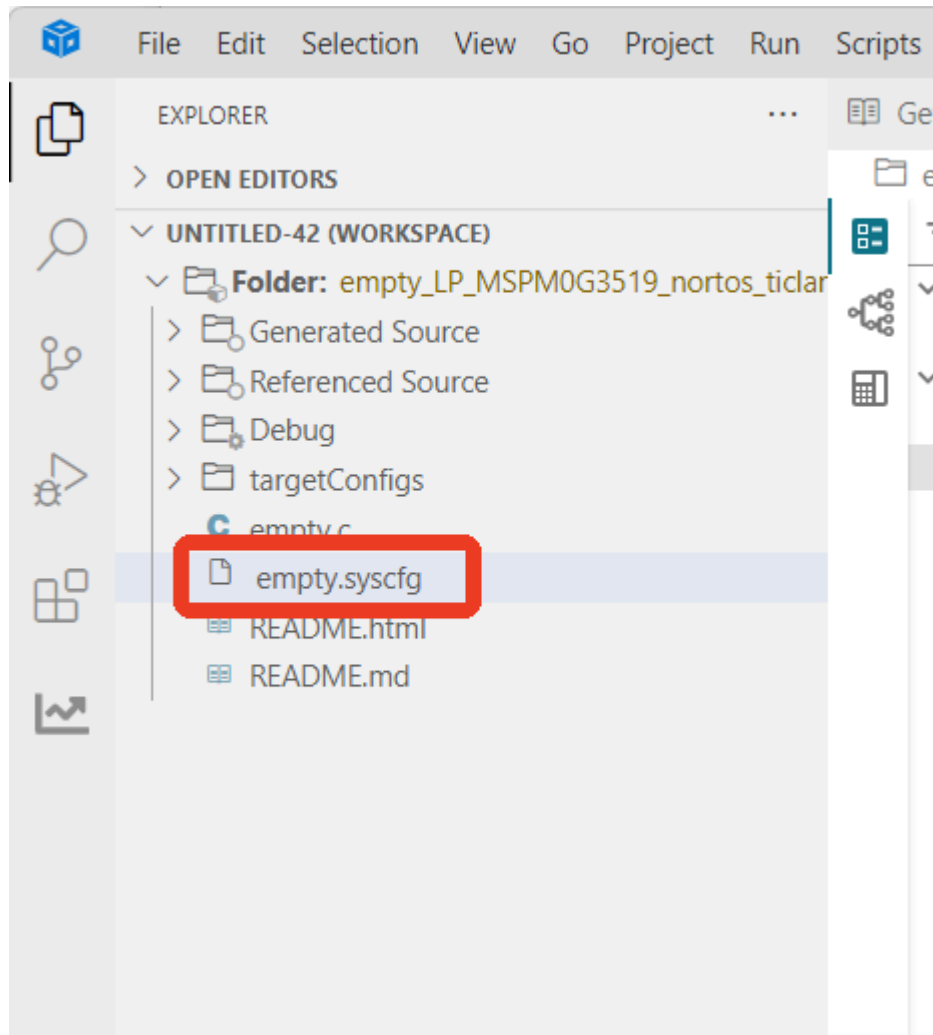
1. Open SYSCONFIG Configuration Tool

Create a new empty project empty in CCS.

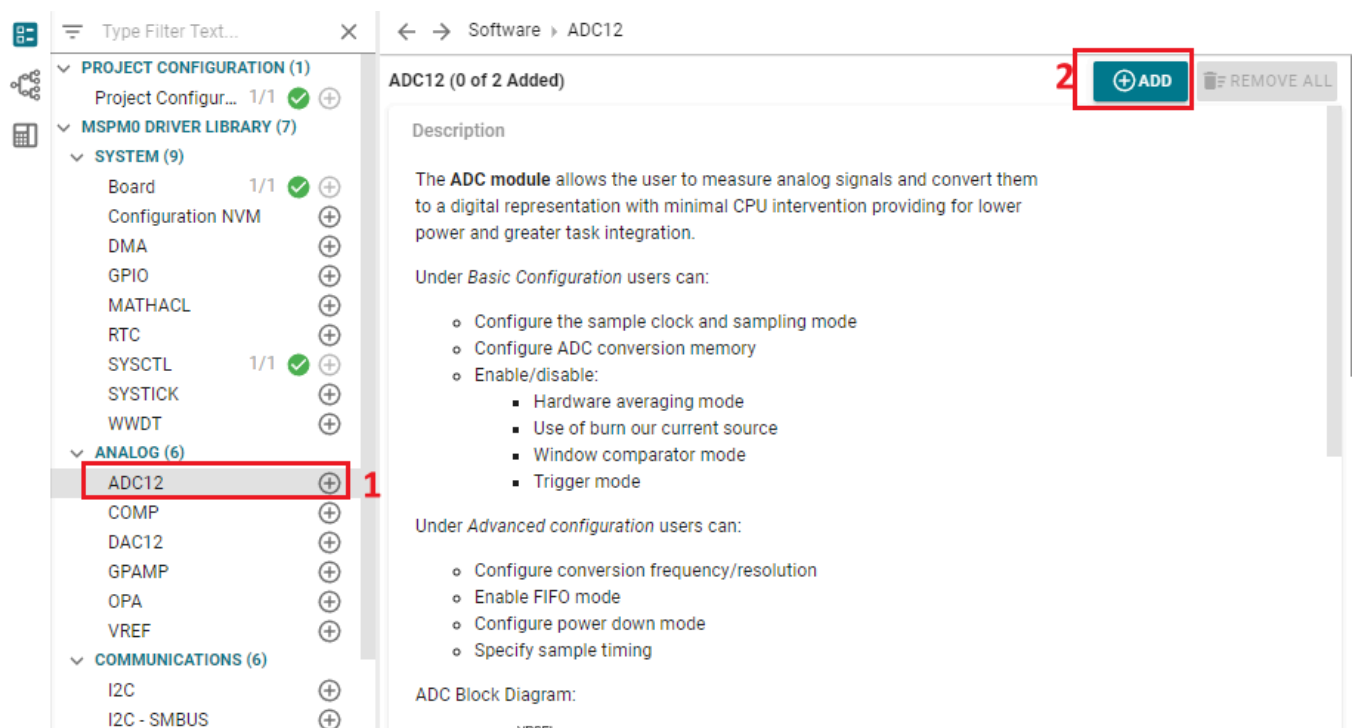




Find and open the empty.syscfg file in the left workspace of CCS.



In SYSCONFIG, select MCU peripherals on the left, find the ADC12 option and click to enter. Click ADD in ADC12 to add an ADC peripheral.



2. ADC Parameter Configuration

The configuration here uses 32MHz ADC frequency, 4MHz sample rate, single repeated conversion mode, software-triggered ADC start sampling, and binary right-aligned data format.

ADC12 (1 of 2 Added) ⓘ

+ ADD

REMOVE ALL

✓ ADC_VOLTAGE

Name

ADC_VOLTAGE

Selected Peripheral

ADC0

Quick Profiles

^

ADC12 Profiles

Custom

Basic Configuration

^

Sample Clock Configuration

^

ADC Clock Source

SYSOSC

ADC Clock Frequency

32.00 MHz

Force SYSOSC Base Freq In STOP

☐

Force SYSOSC Base Freq In RUN

☐

Sample Clock Divider

Divide by 8

Calculated Sample Clock Frequency

4.00 MHz

Sampling Mode Configuration

^

Conversion Mode

Single

Conversion Starting Address

0

Enable Repeat Mode

☒

Sampling Mode

Auto

Trigger Source

Software

Conversion Data Format

Binary unsigned, right aligned

ADC Conversion Memory Configurations

▼

Parameter description:

ADC Clock Source: ADC clock source. Set to `SYSOSC(32MHz)`.

ADC Clock Frequency: Shows the current ADC clock frequency.

Sample Clock Divider: Sample divider. Set to 8 divider.

Calculated Sample clock Frequency: Shows the sample frequency after division.

Conversion Mode: Conversion mode. Set to `Single`, single.

Conversion Starting Address: Conversion starting address. Configure to start sampling from the 0th (related to the storage register later).

Enable Repeat Mode: Whether to enable repeat conversion. Checked here, enabled.

Sampling Mode: Sampling mode. Set to `Auto`, automatic.

Trigger Source: ADC trigger mode. Configure as `Software` software trigger mode.

Conversion Data Format: ADC data conversion format. Configure as `Binary unsigned, right aligned` binary format right-aligned.

MSPM0G3519's ADC supports multiple data alignment methods to adapt to different application scenarios. Common data alignment methods include **left-aligned** and **right-aligned**.

- **Right-aligned mode:** In right-aligned mode, ADC data is right-aligned to the lowest bit after conversion, with insufficient bits filled with 0 in the high bits. Right-aligned mode provides better dynamic range without precision loss.

Rule group data

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
---	---	---	---	-----	-----	----	----	----	----	----	----	----	----	----	----

Injection group data

Sign	Sign	Sign	Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
------	------	------	------	-----	-----	----	----	----	----	----	----	----	----	----	----

DAL=0

- **Left-aligned mode:** In left-aligned mode, ADC data is left-aligned to the highest bit, with insufficient bits filled with 0 in the low bits. Left-aligned mode can improve resolution but will cause reduced dynamic range.

Rule group data

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
---	---	---	---	-----	-----	----	----	----	----	----	----	----	----	----	----

Injection group data

Sign	Sign	Sign	Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
------	------	------	------	-----	-----	----	----	----	----	----	----	----	----	----	----

DAL=0

We configured as right-aligned mode, so we can directly perform operations on the converted data.

3. ADC Acquisition Channel Configuration

ADC Conversion Memory Configurations ^

Active Memory Control BlocksADC Conversion Memory 0 ▾

ADC Conversion Memory 0 Configuration ^

Name	ADC_CH0
Input Channel	Channel 0 ▾
Device Pin Name	PA27
Reference Voltage	VDDA ▾
VDDA	3.30 V
Sample Period Source	Sampling Timer 0 ▾
ADC Conversion Period	102.25 μ s

Optional Configuration ▾

Parameter description:

Active Memory Control Blocks: ADC data storage address. Store ADC conversion result in register 0.

Input Channel: Input channel. Select channel 0 (each channel corresponds to a different pin).

Device Pin Name: Pins automatically selected based on channel. Channel 0's pin is PA27.

Reference Voltage: Voltage reference. Configure to use MCU power VDDA.

ADC Conversion Period: Shows the time for ADC to convert a single data.

Advanced Configuration ^

Total Conversion Frequency	9.78 kHz i When Power Down Mode is set to Auto, ADC wakeup time may need to be considered in each sample window. Refer to the device specific data sheet for specifications on the ADC Wakeup Time.
Conversion Resolution	12-bits ▾
Enable FIFO Mode	☐
Power Down Mode	Auto ▾
Desired Sample Time 0	100 μ s
Actual Sample Time 0	100.00 μ s
Desired Sample Time 1	0 ms
Actual Sample Time 1	0.00 s

Interrupt Configuration ^

Enable Interrupts	MEM0 result loaded interrupt ▾
Interrupt Priority	Default ▾

Parameter description:

Conversion Resolution: Resolution configured as 12-bits.

Desired Sample Time 0: Sample time set to 10ms.

Enable interrupts: Enable ADC acquisition complete interrupt. When ADC acquisition completes, it triggers an interrupt, and we perform data processing.

PinMux Peripheral and Pin Configuration		^
ADC12 Peripheral	ADC0	▼ 🔒
ADC12 Channel 0 Pin	PA27/31	▼ 🔒

4. Serial Parameter Configuration

This case outputs ADC acquired values to the upper computer (computer) display via serial port, so serial port sending function also needs to be configured. The not shown part is the default option.

Name	UART_0
Selected Peripheral	UART0

Quick Profiles

UART Profiles

Custom

Basic Configuration
UART Initialization Configuration

Clock Source	BUSCLK
Clock Divider	Divide by 1
Calculated Clock Source	32.00 MHz
Target Baud Rate	9600
Calculated Baud Rate	9600.24
Calculated Error (%)	0.0025
Word Length	8 bits
Parity	None
Stop Bits	One
HW Flow Control	Disable HW flow control

Advanced Configuration

UART Mode ⓘ	Normal UART Mode
Communication Direction	TX and RX
Oversampling	16x
Enable FIFOs	☐
Analog Glitch Filter	Disabled
Digital Glitch Filter	0
Calculated Digital Glitch Filter	0.00 s
RX Timeout Interrupt Counts	0
Calculated RX Timeout Interrupt	0.00 s
Enable Internal Loopback	☐
Enable Majority Voting	☐
Enable MSB First	☐

PinMux Peripheral and Pin Configuration

UART Peripheral	Any(UART0)
RX Pin	PA11/57
TX Pin	PA10/56

Save the configuration in the .syscfg file using the shortcut key Ctrl+S.

5. Write Program

In the empty.c file, write the following code

```
#include "ti_msp_dl_config.h"
#include "stdio.h"

volatile bool gCheckADC;          //ADC acquisition success flag
unsigned int adc_getValue(void); //Read ADC data

//Send string via serial port
void uart0_send_string(char* str)
{
    // The current string address is not at the end and the string's first address is not empty
    while(*str!=0&&str!=0)
    {
        // wait when serial port 0 is busy, and send the incoming characters when it is not busy
        while( DL_UART_isBusy(UART_0_INST) == true );
        // Send the characters in the first address of the string, and increment the first address after sending.
        DL_UART_Main_transmitData(UART_0_INST, *str++);
    }
}

int main(void)
{
    char output_buff[50] = {0};
    unsigned int adc_value = 0;
    float voltage_value = 0;

    SYSCFG_DL_init();

    // Enable ADC interrupt
    NVIC_EnableIRQ(ADC_VOLTAGE_INST_INT_IRQN);

    uart0_send_string("adc Demo start\r\n");
    while (1)
    {
        // Get ADC data
        adc_value = adc_getValue();
        sprintf(output_buff, "adc value:%d\r\n", adc_value);
        uart0_send_string(output_buff);

        // Convert the data collected by ADC into voltage
        voltage_value = adc_value/4095.0*3.3;
        sprintf(output_buff, "voltage value:%.2f\r\n", voltage_value);
        uart0_send_string(output_buff);

        delay_cycles(32000000);
    }
}
```

```

    }
}

// Read ADC data
unsigned int adc_getValue(void)
{
    unsigned int gAdcResult = 0;

    // Software triggers ADC to start conversion
    DL_ADC12_startConversion(ADC_VOLTAGE_INST);
    // If the current state is in transition, wait for the transition to end.
    while (false == gCheckADC) {
        __WFE();
    }
    // Get data
    gAdcResult = DL_ADC12_getMemResult(ADC_VOLTAGE_INST, ADC_VOLTAGE_ADCMEM_ADC_CH0);

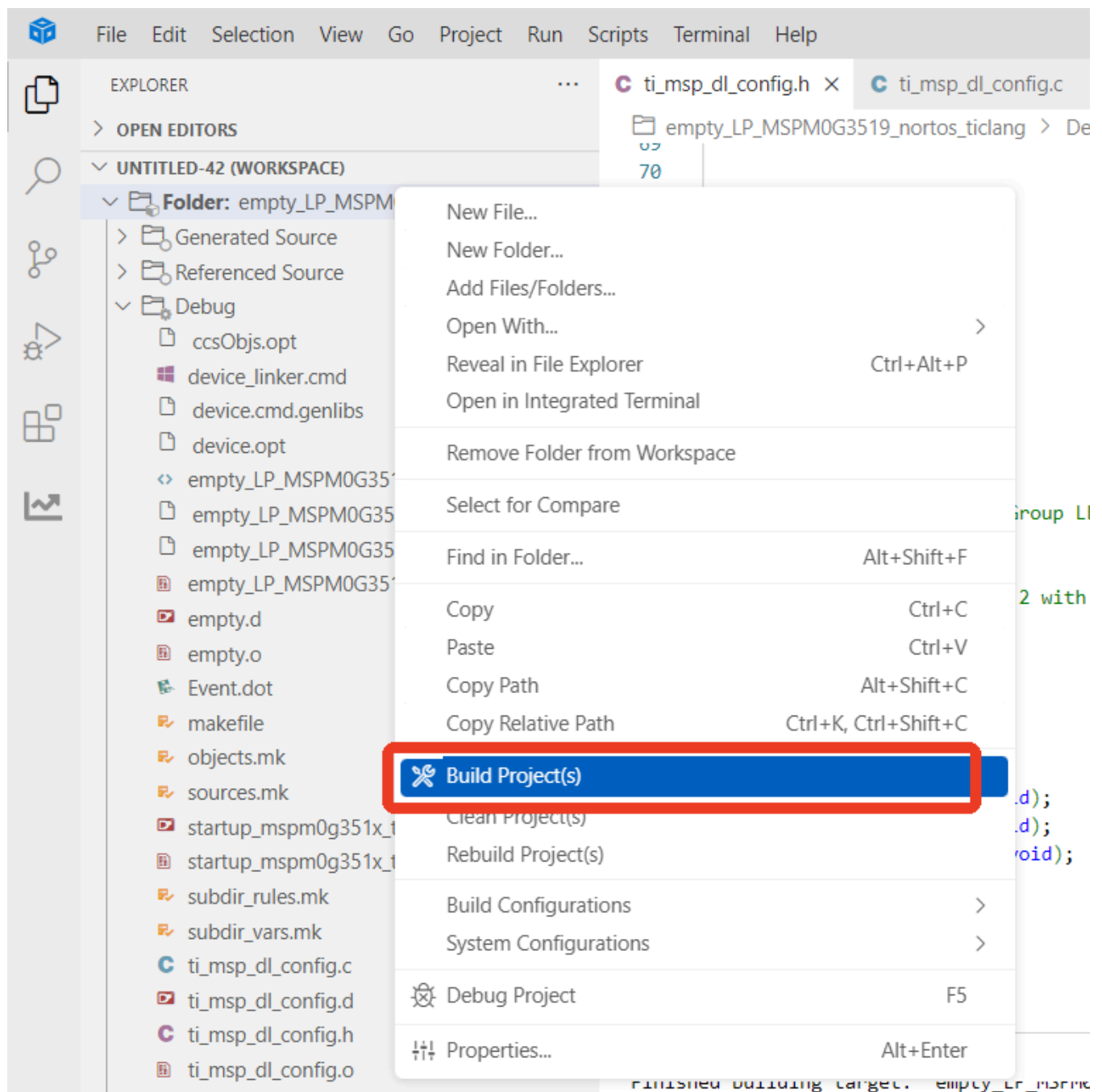
    // Clear flag
    gCheckADC = false;

    return gAdcResult;
}

// ADC interrupt service function
void ADC_VOLTAGE_INST_IRQHandler(void)
{
    // Query and clear ADC interrupt
    switch (DL_ADC12_getPendingInterrupt(ADC_VOLTAGE_INST))
    {
        // Check whether data collection is completed
        case DL_ADC12_IIDX_MEM0_RESULT_LOADED:
            gCheckADC = true; // Set the flag position to 1
            break;
        default:
            break;
    }
}
}

```

6. Compile



Compilation was successful. You can now download the program to the development board.

4. Program Analysis

- empty.c

```

7  //串口发送字符串 Send string via serial port
8  void uart0_send_string(char* str)
9  {
10     //当前字符串地址不在结尾 并且 字符串首地址不为空
11     // The current string address is not at the end and the string's first address is not empty
12     while(*str!=0&&str!=0)
13     {
14         //当串口0忙的时候等待, 不忙的时候再发送传进来的字符
15         // Wait when serial port 0 is busy, and send the incoming characters when it is not busy
16         while( DL_UART_isBusy(UART_0_INST) == true );
17         //发送字符串首地址中的字符, 并且在发送完成之后首地址自增
18         // Send the characters in the first address of the string, and increment the first address after sending.
19         DL_UART_Main_transmitData(UART_0_INST, *str++);
20     }
21 }

```

The `uart0_send_string` function sends a string through UART serial port. It iterates through each character with `while (*str != 0)` until the string ends (`*str == 0`). Before sending each character, it checks if UART is busy via `DL_UART_isBusy(UART_0_INST)`. Data can only be sent when UART is idle. It sends the current character via `DL_UART_Main_transmitData(UART_0_INST, *str++)` and increments `str` to point to the next character.

```

85  //读取ADC的数据 Read ADC data
86  unsigned int adc_getValue(void)
87  {
88      unsigned int gAdcResult = 0;
89
90      //软件触发ADC开始转换 Software triggers ADC to start conversion
91      DL_ADC12_startConversion(ADC_VOLTAGE_INST);
92      //如果当前状态为正在转换中则等待转换结束
93      // If the current state is converting, wait for the conversion to end
94      while (false == gCheckADC) {
95          __WFE();
96      }
97      //获取数据 Get data
98      gAdcResult = DL_ADC12_getMemResult(ADC_VOLTAGE_INST, ADC_VOLTAGE_ADCMEM_ADC_CH0);
99
100     //清除标志位 Clear flag
101     gCheckADC = false;
102
103     return gAdcResult;
104 }

```

This code reads ADC (analog-to-digital converter) data. It triggers ADC conversion, waits for conversion to complete, then gets the ADC sample result and returns it.

```

115 //ADC中断服务函数 ADC interrupt service function
116 void ADC_VOLTAGE_INST_IRQHandler(void)
117 {
118     //查询并清除ADC中断 Query and clear ADC interrupt
119     switch (DL_ADC12_getPendingInterrupt(ADC_VOLTAGE_INST))
120     {
121         // 检查是否完成数据采集 Check whether data collection is completed
122         case DL_ADC12_IIDX_MEM0_RESULT_LOADED:
123             gCheckADC = true; //将标志位置1 Set the flag position to 1
124             break;
125         default:
126             break;
127     }
128 }

```

The ADC interrupt service function `ADC_VOLTAGE_INST_IRQHandler` is used to handle interrupt events and set the flag when ADC data acquisition is complete.

```

23 int main(void)
24 {
25     char output_buff[50] = {0};
26     unsigned int adc_value = 0;
27     float voltage_value = 0;
28
29     SYSCFG_DL_init();
30
31     //开启ADC中断 Enable ADC interrupt
32     NVIC_EnableIRQ(ADC_VOLTAGE_INST_INT_IRQN);
33
34     uart0_send_string("adc Demo start\r\n");
35     while (1)
36     {
37         //获取ADC数据 Get ADC data
38         adc_value = adc_getValue();
39         sprintf(output_buff, "adc value:%d\r\n", adc_value);
40         uart0_send_string(output_buff);
41
42         //将ADC采集的数据换算为电压 Convert the data collected by ADC into voltage
43         voltage_value = adc_value/4095.0*3.3;
44         sprintf(output_buff, "voltage value:%.2f\r\n", voltage_value);
45         uart0_send_string(output_buff);
46
47         delay_cycles(32000000);
48     }
49 }

```

This code implements reading ADC (analog-to-digital converter) data, calculating voltage values, and outputting results through UART serial port. Initialize system configuration and ADC interrupts. Enter main loop, continuously read ADC data and calculate voltage. Send ADC values and calculated voltage through UART serial port to terminal.

5. Experimental Phenomena

After the program is downloaded, after configuring the serial assistant as shown in the figure below, adjust the potentiometer knob. You can see the voltage changing in real-time between 0-3.3v in the serial assistant receive window.

